

KsqlDB et Confluent

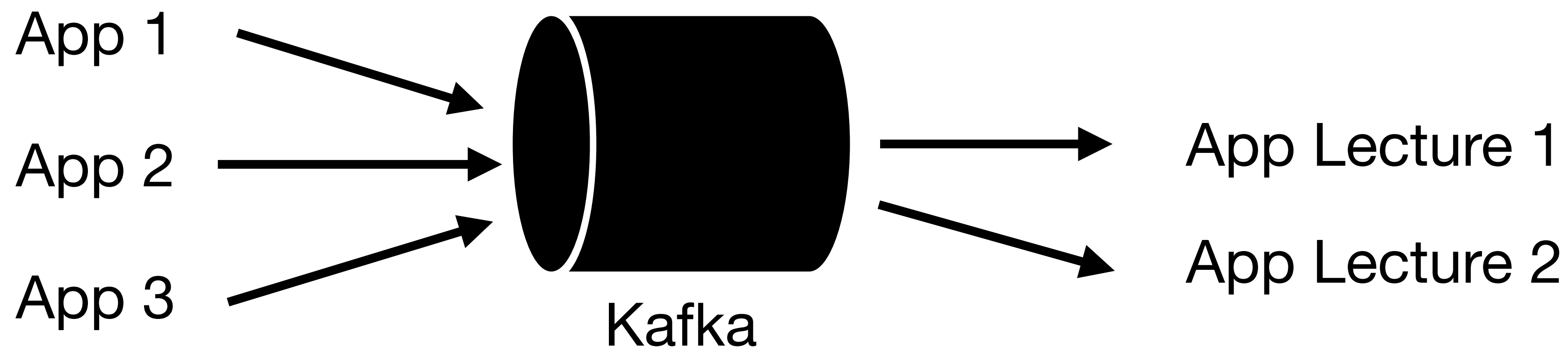
Lionel SOUOP

Kafka

Introduction

Kafka est un système de messagerie open source qui permet à plusieurs applications de communiquer, de s'échanger des messages entre elles via des flux de données.

Kafka s'imaginer comme une queue dans laquelle des messages peuvent être lus et écrits:

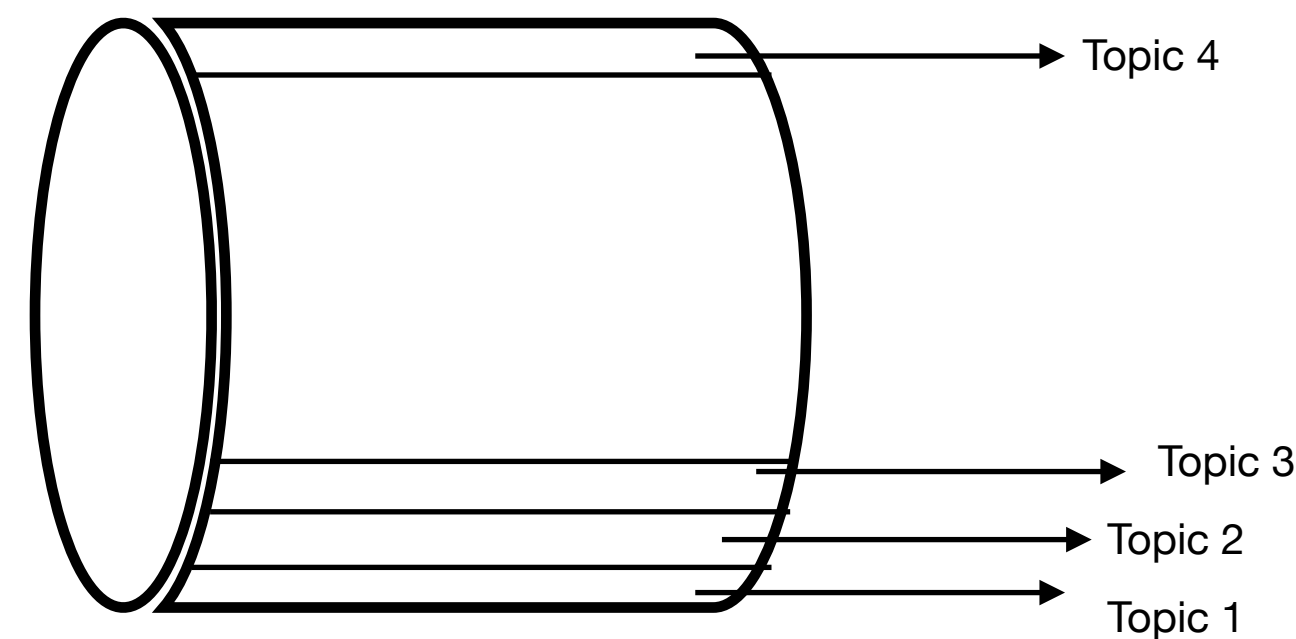


Kafka

Introduction

L'infrastructure Kafka peut avoir plusieurs topics.

Un topic est une abstraction qui permet de catégoriser des messages dans Kafka, et une application qui lit ou écrit dans(de) Kafka vise toujours un topic particulier.



Kafka Queue

Kafka

Quelques détails

Kafka est conçu pour :

- Gérer de très grandes quantités de données.
- Assurer une transmission rapide et fiable des données.
- Permettre à plusieurs applications de lire les mêmes données simultanément.

Kafka est une infrastructure Big Data de traitement des données en streaming et permet de faire du temps réel

Kafka

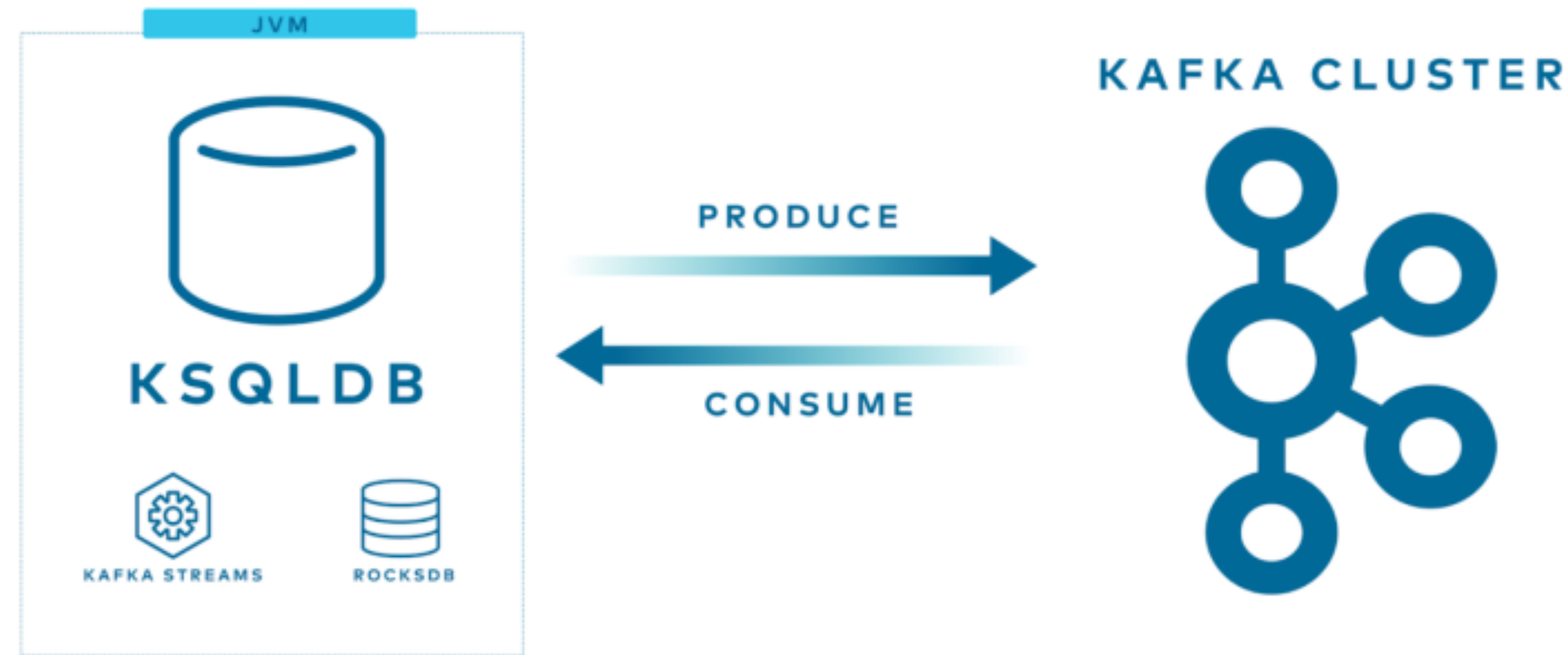
Les outils similaires

- RabbitMQ
- Les clouds providers et leurs solutions de streaming
 - AWS - SQS, Kinesis, Lambda functions etc ...
 - GCP - pub/sub, BigQuery etc ...
 - Azure - Events Hub etc ...

KsqIDB

KsqlDB

Introduction



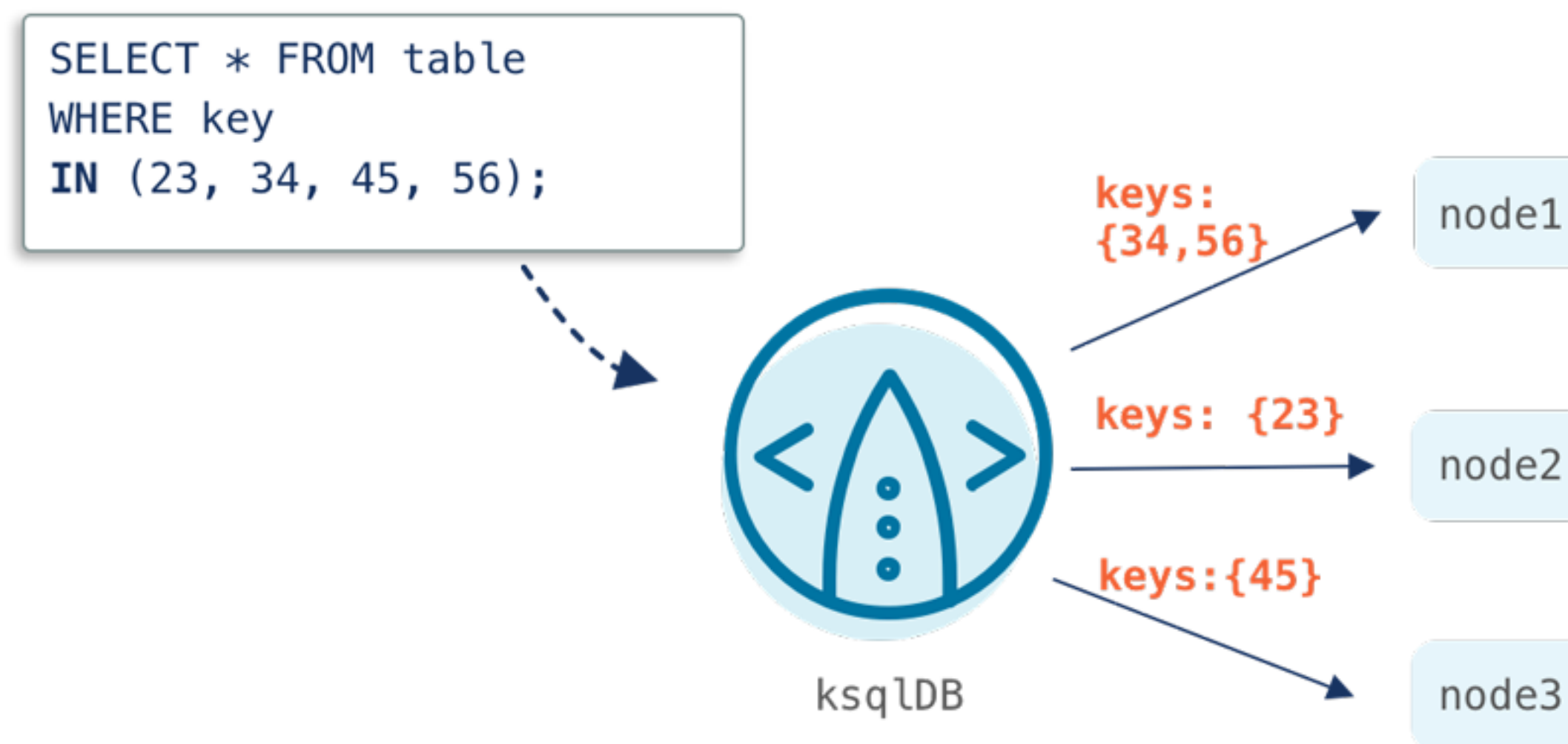
- KsqlDB est un système distribué qui fonctionne en s'appuyant sur Kafka comme outil de stockage
- ksqlDB est une base de données de streaming conçue pour faciliter la création d'applications de traitement de flux en temps réel sur Apache Kafka

KsqlDB

SQL

Traitement de flux avec SQL:

- ksqlDB permet d'interroger et de transformer des flux de données en temps réel en utilisant une syntaxe SQL familière.
- Au lieu de travailler avec du code complexe, vous pouvez utiliser des requêtes SQL pour analyser, manipuler et transformer les données en streaming.



KsqlDB

Utilisation

Applications de streaming en temps réel:

- ksqlDB est idéal pour créer des applications qui réagissent instantanément aux événements, comme la détection d'anomalies, la surveillance de transactions ou l'analyse de données de capteurs.
- Il permet de créer des flux et des tables matérialisées à partir de topics Kafka.
- Il prend en charge les opérations de jointure, d'agrégation et de fenêtrage sur les flux de données.

KsqlDB - Opérations

KsqlDB

Creation d'un stream

Dans ksqlDB, un STREAM est une structure représentant un flux de données en temps réel provenant d'un topic Kafka. Il s'agit d'une suite continue d'événements (messages) qui sont immuables et ne sont jamais mis à jour ni supprimés.

Un STREAM est assimilable à une table append-only où chaque nouvel événement est ajouté en continu.

KsqlDB

Creation d'un stream

```
CREATE STREAM transactions_stream (  
  id STRING,  
  montant DOUBLE,  
  type STRING,  
  timestamp STRING  
) WITH (  
  KAFKA_TOPIC='transactions',  
  VALUE_FORMAT='JSON'  
);
```

Ce STREAM lit les événements d'un topic Kafka nommé **transactions** et interprète les données au format JSON.

KsqlDB

Lecture d'un stream

```
SELECT * FROM transactions_stream EMIT CHANGES;
```

Affiche les nouveaux événements au fur et à mesure qu'ils arrivent.

KsqlDB

Filtre sur un stream

```
SELECT * FROM transactions_stream WHERE montant > 100 EMIT CHANGES;
```

Filtre les transaction dont le montant est supérieur à 100

Il est possible de créer un autre stream à partir d'un stream existant. Dans l'exemple ci-dessous, les transactions importantes sont écrites dans un nouveau topic Kafka, automatiquement créé.

```
CREATE STREAM transactions_importantes AS  
SELECT * FROM transactions_stream WHERE montant > 100;
```


KsqlDB

Merge streams

KsqlDB permet de fusionner plusieurs streams en un seul en utilisant:

- Union pour concatener plusieurs streams
- Join pour enrichir un flux avec des données d'un autre.

KsqlDB

Merge streams - Union

```
CREATE STREAM transactions_A AS
SELECT * FROM transactions_stream WHERE type = 'Achat';

CREATE STREAM transactions_R AS
SELECT * FROM transactions_stream WHERE type = 'Retrait';

CREATE STREAM all_transactions AS
SELECT * FROM transactions_A
UNION ALL
SELECT * FROM transactions_R;
```

Cela crée un nouveau flux all_transactions combinant les achats et retraits.

Différence entre UNION et UNION ALL

- UNION : Élimine les doublons.
- UNION ALL : Garde tous les événements, même en double.

KsqlDB

Merge streams - Join

Si vous souhaitez combiner deux flux en croisant les données (ex : fusionner transactions et détails clients)

Types de JOIN possibles dans ksqlDB

- INNER JOIN : Garde uniquement les correspondances.
- LEFT JOIN : Garde tous les événements du STREAM principal, même sans correspondance.

Le STREAM ci-dessous fusionne les transactions avec les données utilisateur en fonction de user_id

```
CREATE STREAM transactions_users AS
SELECT t.id, t.montant, t.type, u.nom, u.email
FROM transactions_stream t
INNER JOIN users_stream u
ON t.user_id = u.id;
```

KsqlDB

Split Streams

Pourquoi diviser un Stream?

- ✅ Filtrage des événements : Séparer les données en plusieurs catégories.
- ✅ Optimisation des performances : Traiter chaque flux séparément.
- ✅ Simplification de l'analyse : Organiser les données selon des critères métier.

Par exemple, nous voulons séparer les transactions en trois flux distincts :

1. Achats (transactions_achat)
2. Retraits (transactions_retrait)
3. Virements (transactions_virement)

```
CREATE STREAM transactions_achat AS
SELECT * FROM transactions_stream WHERE type = 'Achat';

CREATE STREAM transactions_retrait AS
SELECT * FROM transactions_stream WHERE type = 'Retrait';

CREATE STREAM transactions_virement AS
SELECT * FROM transactions_stream WHERE type = 'Virement';
```

KsqlDB

Windowing

Le fenêtrage (Windowing) dans ksqlDB permet d’agréger des événements sur une période de temps définie. Il est essentiel pour analyser des données en streaming et appliquer des calculs sur des intervalles temporels, comme :

- Nombre de transactions par minute
- Moyenne des ventes toutes les 10 minutes
- Détection de fraudes en fonction des transactions récentes

ksqlDB propose **trois types de fenêtres** pour gérer les flux de données en temps réel :

Type de fenêtre	Description	Exemple
TUMBLING	Fenêtres fixes et non chevauchantes de durée égale.	Total des ventes toutes les 5 minutes.
HOPPING	Fenêtres glissantes avec chevauchement , définies par une durée et un intervalle de saut.	Moyenne des transactions toutes les 10 min avec un pas de 5 min.
SESSION	Fenêtres dynamiques basées sur l'activité , qui ferment après un certain temps d'inactivité.	Grouper les actions d'un utilisateur jusqu'à 30 min d'inactivité.

KsqlDB

Windowing - Tumbling

- Crée des fenêtres de 5 minutes et compte les transactions pour chaque type (Achat, Retrait, etc.)

```
SELECT type, COUNT(*) AS nombre_transactions
FROM transactions_stream
WINDOW TUMBLING (SIZE 5 MINUTES)
GROUP BY type
EMIT CHANGES;
```

KsqlDB

Windowing - Hopping

- Crée une fenêtre de 10 minutes, avancée toutes les 5 minutes (chevauchement partiel).

```
SELECT type, AVG(montant) AS moyenne_montant
FROM transactions_stream
WINDOW HOPPING (SIZE 10 MINUTES, ADVANCE BY 5 MINUTES)
GROUP BY type
EMIT CHANGES;
```

KsqlDB

Windowing - Session

- Groupe les transactions d'un utilisateur tant qu'il reste actif dans une fenêtre de 30 minutes.

```
SELECT user_id, COUNT(*) AS nombre_transactions
FROM transactions_stream
WINDOW SESSION (30 MINUTES)
GROUP BY user_id
EMIT CHANGES;
```


KsqlDB

Windowing - cas d'usage

- Analyse en temps réel des ventes par période - nombre de ventes lors des 5 dernières minutes
- Détection de fraudes en identifiant des pics d'activité suspects.
- Optimisation des recommandations en suivant l'activité récente d'un utilisateur - plus besoin d'utiliser des ressources pour calculer les recommandations d'un utilisateur hors session.

KsqlDB

Tables

Les streams et les tables sont des façon de gérer des streams dans KsqlDB.

Les streams représentent une suite continue d'éléments d'éléments

Les tables ne stockent que la valeur la plus récente d'une clé

PS: Pour parler de table, il faut avoir bien sur une clé

La création et la manipulation des tables est similaire aux streams

```
CREATE TABLE comptes (  
    user_id STRING PRIMARY KEY,  
    solde DOUBLE  
) WITH (  
    KAFKA_TOPIC='comptes_topic',  
    VALUE_FORMAT='JSON'  
);
```


KsqlDB

SQL request

Dans ksqlDB, les requêtes sont divisées en deux catégories principales : les requêtes push et les requêtes pull. Chacune répond à des besoins différents en matière d'interrogation de flux de données.

Requêtes Push

- Les requêtes push établissent une connexion continue avec ksqlDB.
- Elles "poussent" les résultats vers le client en temps réel, à mesure que de nouvelles données arrivent dans les flux ou les tables.
- Utilisation de la clause **EMIT CHANGES**.

```
SELECT * FROM transactions_stream EMIT CHANGES;
```

Affiche les nouvelles mises à jour des soldes de comptes en temps réel.

KsqlDB

SQL request

Requêtes Pull

- Les requêtes pull interrogent l'état actuel d'une table matérialisée à un instant donné.
- Elles renvoient un ensemble de résultats unique et terminé.
- Utiles pour obtenir un instantané des données.

```
SELECT * FROM comptes WHERE user_id = 'U001';
```

Retourne uniquement l'état actuel du compte U001.

KsqlDB

Lambda functions

Les fonctions lambda dans ksqlDB permettent d'appliquer des transformations complexes sur des TABLEs et STREAMs en utilisant des fonctions **anonymes**. Elles sont souvent utilisées avec des fonctions comme TRANSFORM, REDUCE, FILTER, et MAP. Elles s'appliquent généralement sur les champs de type liste ou map(dictionnaire)

Syntaxe:

(param) => expression

param est le nom du paramètre et expression est l'opération effectuée sur ce paramètre.

Exemple:

(x) => x * 2

Cette fonction double la valeur de x

KsqlDB

Lambda functions

Exemple 1 : Appliquer FILTER pour garder seulement les transactions > 100€

```
SELECT FILTER(ARRAY[50, 120, 30, 200], x => x > 100);
```

FILTER: Le type de fonction

ARRAY[50, 120, 30, 200]: élément sur lequel appliquer la fonction

x => x > 100: La fonction à appliquer

KsqlDB

Lambda functions

Exemple 2 : Transformer une liste avec MAP

```
SELECT MAP(ARRAY[1, 2, 3, 4], x => x * 10);
```

Exemple 3 : Réduire un tableau avec REDUCE (somme totale)

```
SELECT REDUCE(ARRAY[5, 15, 20], 0, (x, acc) => x + acc);
```

0 est la valeur initiale de l'accumulateur

Avantages et limitations

KsqlDB

Avantages

1 SQL pour le Streaming

- Permet de manipuler des flux Kafka avec des requêtes SQL classiques (SELECT, JOIN, GROUP BY).
- Facile à apprendre pour ceux qui connaissent déjà SQL.

2 Traitement d'événements en Temps Réel et de construire des pipeline complexes de données

3 Vue Matérialisée et Requetes Instantanées

- Supporte les TABLEs pour stocker l'état actuel des données (similaire à une base de données NoSQL).
- Supporte les Pull Queries pour récupérer instantanément une valeur actuelle dans une TABLE.

4 Évolutivité et Intégration Kafka-Native

- S'intègre parfaitement avec Kafka Streams, Connectors, et les outils de l'écosystème Kafka.
- Peut gérer de gros volumes de données et s'adapte facilement à des architectures distribuées.

5 Pas Besoin de Développement Complexe

- Évite d'avoir à écrire du code Java/Scala avec Kafka Streams.
- Accélère le développement des applications data-driven en entreprise.

KsqlDB

Limitation

1 Limitations sur les Requêtes SQL

- Ne supporte pas certaines fonctionnalités SQL avancées comme les sous-requêtes imbriquées.
- Pas de support natif pour les transactions ACID comme une base de données classique.

2 Coût et Consommation de Ressources

- Peut être gourmand en CPU et mémoire sur des flux à très haut débit.
- Nécessite une infrastructure Kafka bien dimensionnée pour éviter les goulots d'étranglement.

3 Gestion des Données Historisées

- ksqlDB fonctionne mieux en temps réel et n'est pas conçu pour stocker des données historiques sur le long terme.
- Si vous avez besoin d'analyses sur des périodes longues, il faudra exporter les données vers un entrepôt (Snowflake, BigQuery, etc.).

4 Pas Idéal pour des Cas Complexes

- Certaines transformations avancées nécessitent encore Kafka Streams ou des outils comme Flink.
- Ne permet pas de gérer des workflows complexes avec plusieurs étapes dépendantes.